Dizionari

Esercizi 0 (riscaldamento)

- (*) Scrivere una funzione rimuovi_chiavi(D, lista_chiavi) che prende in input un dizionario e una lista di chiavi. La funzione deve modificare il dizionario, rimuovendo tutte le chiavi in lista_chiavi, se esistono.
- (*) Scrivere una funzione azzera_valori(D, lista_chiavi) che prende in input un dizionario e una lista di chiavi. La funzione deve modificare il dizionario, settando a "0" i valori corrispondenti alle chiavi in lista_chiavi, se esistono.
- (*) Scrivere una funzione stesse_chiavi(D1, D2) che prende in input due dizionari e restituisce True se hanno le stesse chiavi.
- (*) Scrivere una funzione stesso_dizionario(D1, D2) che prende in input due dizionari e restituisce True se hanno le stesse chiavi con gli stessi valori (ovvero se sono uguali).

Esercizio 1

- (*) Scrivi una funzione max_key(D) che prende in input un dizionario e ritorna la chiave che è associata al massimo valore.
  - Ad esempio se D = {"everest": 8848, "k2" : 8609, "cervino" : 4478} la funzione deve restituire "everest".
  - Le precondizioni sono le seguenti
    - D sia un dizionario non vuoto
    - i valori siano numeri
    - non ci siano due chiavi associate allo stesso valore massimo (es. due montagne alte come l'everest)

Esercizio 2

- (*) Scrivi una funzione merge_dict(D1, D2) che resitiuisce un dizionario unico a partire da D1 e D2. In caso di chiavi duplicate, il risultato deve contenere il valore massimo tra i due.
  - Ad esempio se D1 = {"everest": 8848, "k2" : 8609} e D2 = {"everest": 8840, "cervino" : 4478} la funzione deve restituire {"everest": 8848, "k2" : 8609, "cervino" : 4478}.
  - Le precondizioni sono le seguenti
    - D1, D2 sono dizionari o i valori sono numeri

Esercizio 3

- (**) Scrivi una funzione merge_dict_noloss(D1, D2) che resitiuisce un dizionario unico a partire da D1 e D2. In caso di chiavi duplicate, il risultato deve contenere entrambi i valori riportati nei due dizionari.
  - Ad esempio se D1 = {"everest": 8848, "k2" : 8609} e D2 = {"everest": 8840, "cervino" : 4478} la funzione deve restituire {"everest": [8848, 8840], "k2" : [8609], "cervino" : [4478]}.
  - Le precondizioni sono le seguenti
    - D1, D2 sono dizionari

Esercizio 4

- (**) Scrivi una funzione invert(D) che resitiuisce un dizionario dove le chiavi sono i valori di D e i valori sono la lista di chiavi a cui sono associati in D.
  - Ad esempio se D = {"everest": 8848, "k2": 8609, "cervino": 4478, "montagnaX": 4478} la funzione deve restituire {8848: ["everest"], 8609: ["k2"], 4478: ["cervino", "montagnaX"]}.
  - Le precondizioni sono le seguenti
    - D è un dizionario e i valori sono numeri o stringhe

File

Esercizi 0 (riscaldamento)

- (*) Scrivere una funzione salva_dizionario(D, filepath) che salvi il contenuto di un certo dizionario in un file. Si usi ascii come encoding e si scriva una coppia chiave valore per riga, separando la chiave dal valore con il simbolo speciale ",". Si può assumere che il simbolo speciale non sia conenuto né nelle chiavi né nei valori e che il dizionario non contenga altre strutture dati, ovvero che chiavi e valori siano o stringhe o numeri.
  - Ad esempio se D={"luca" : 3336678, "marta": 3498812, "tom": 5672234} il file conterrà le seguenti tre righe
    - luca, 3336678
    - marta, 3498812
    - tom, 5672234
- (*) Scrivere una funzione carica_dizionario(filepath) che restituisce un dizionario, precedentemente salvato in un file con la funzione salva_dizionario.

Esercizi 1

- (*) Scrivere una funzione copia(filepath_from, filepath_to, encoding='utf-8') che prende in input il percorso identificativo (relativo o assoluto) di due file e copia il conenuto dell'uno nell'altro.
  - Inoltre la funzione
    - restituisce TypeError se i due parametri non sono stringhe
    - restituisce ValueError se il primo path non esiste

- Per controllare che un percorso esista si può usare il metodo os.path.exists(path) della libreria os

Esercizi 2

- (*) Scrivere una funzione ricerca_linee(filepath, stringa, encoding='utf-8') che restituisce una lista di tutte le righe che contengono la stringa in input. Nella lista si usi il numero di riga, ad esempio, se la stringa è contenuta solo nella prima e nella terza riga, si restituisca la stringa [1, 3].
  - Inoltre la funzione
    - restituisce TypeError se i parametri non sono del tipo opportuno
    - restituisce ValueError se il path non esiste

- Per controllare che una stringa x sia contenuta in un'altra y si usi il metodo y.find(x).

Note

- Gli esercizi che seguono hanno l'obiettivo di implementare un semplice correttore automatica di ortografia, usando file e dizionari
- Ad esempio che corregga la stringa "ortograzia" in "ortografia"

Esercizi 3

- (**) Scrivere una funzione word_freq(file_path, encoding='utf-8') che prende in input il percorso di un file e di una codifica di caratteri e restituisce un dizionario con tutte le parole che occorrono nel file e la relativa frequenza. Si legga il file riga per riga e si riusino le funzioni sviluppate nell'esercitazione precedente per ottenere la lista di parole di ogni riga, tenendo in considerazione solo i caratteri alfabetici e considerando equivalenti maiuscoli e minuscoli.
  - Per quanto riguarda il percorso del file, si possono usare sia percorsi relativi che assoluti. Seguono esempi di chiamate valide a word_freq
    - word_freq('documento.txt’) # document.txt è nella cartella corrente
    - word_freq('../didattica/eval.text’) # percorso relativo su Mac e Linux
    - word_freq('/usr/share/dict/README') # percorso assoluto su Mac e Linux
    - word_freq('C:\\Documents\\romanzo.txt') # percorso assoluto su windows
    - word_freq('documento.txt',encoding='Latin1') # encoding diverso dal default
  - Usa come argomenti di test i file di esempio disponibili su moodle in utf-8

Esercizi 4

- (***) Scrivere una funzione find_likely(D, word) che prende in input un dizionario di frequenze e una parola.
  - La funzione deve restituire una parola wordOut che sia
    - tra le chiavi del dizionario D
    - la più simile a word dal punto di vista della scrittura, ad esempio casa è più simile a nasa di quanto lo sia a dimora (anche se dimora è più vicina a casa come significato)
    - a parità di similarità si restituisca quella più frequente (ovvero con più alto valore in D), per verificare l'uguaglianza si usi round() a 3 cifre decimali, ad esempio round(x, 3)
    - a parità di frequenza si resituisca una qualsiasi
  - Usa come argomenti di test i risultati dell'invocazione della funzione precedente
  - Per calcolare il punteggio di similarità tra stringhe, si usi SequenceMatcher(None, stringa1, stringa2).ratio() importando la libreria difflib.

Esercizi 5

- (*) Scrivere una funzione suggest_correction(D, sentence) che prende in input un dizionario di frequenze e una frase
  - La funzione restituisce per ognuna delle parole della frase
    - se è in D, la parola stessa
    - se non lo è, la più verosimile secondo find_likely
  - Ad esempio, data la frase "the cat iis on the tabe", la funzione restituisce "the cat is on the table"
  - Usa la funzione suggest_correction insieme alle precedenti per realizzare un semplice correttore ortografico